WRITEUP CTF: Oopsie

Bernardo Barros Blanco

Abril de 2026

Resumo

Este relatório detalha a metodologia de ataque aplicada à máquina **Oopsie**. A exploração percorre as fases de reconhecimento web, quebra de controle de acesso (IDOR), execução remota de código via Local File Inclusion (LFI) e escalação de privilégios através de manipulação de variáveis de ambiente (*Path Injection*).

1 Reconhecimento e Enumeração

A fase inicial consistiu em mapear a superfície de ataque do servidor alvo. Com o túnel VPN estabelecido, iniciamos a enumeração de diretórios para encontrar pontos de entrada ocultos.

```
# Comando utilizado para descobrir pastas no servidor web
gobuster dir -u http://10.129.126.111/ -w /usr/share/wordlists/dirb/
    common.txt
```

Listing 1: Enumeração de diretórios com Gobuster

O `Gobuster` revelou o diretório `/cdn-cgi/login/`, que nos levou a uma interface de autenticação.

2 Exploração de Vulnerabilidades Web

2.1 Broken Access Control (IDOR)

Ao acessar como convidado (*guest*), observamos que o sistema gerenciava sessões através de cookies. Ao analisar a página de contas, notamos que o parâmetro `id` na URL era vulnerável a **IDOR** (Insecure Direct Object Reference). Ao iterar os IDs, descobrimos os dados do administrador:

- **Access ID do Admin:** 34322
- **Role:** admin

Manipulando os cookies no navegador para refletir esses valores, ganhamos acesso à página de *Uploads*.

2.2 Local File Inclusion (LFI) e RCE

Realizamos o upload de um script PHP malicioso (`exploit.php`) projetado para executar comandos no sistema operacional. Para disparar o exploit, utilizamos uma falha de inclusão de arquivo no parâmetro `content`:

```
curl "http://10.129.126.111/cdn-cgi/login/admin.php?content=../uploads/
    exploit.php&cmd=cat%20../db.php"
```

Listing 2: Execução de comando via LFI

Este comando revelou o arquivo de configuração do banco de dados, onde encontramos a senha do usuário **robert**.

3 Movimentação Lateral e Escalação de Privilégios

3.1 Acesso SSH

Com as credenciais obtidas, estabelecemos uma conexão segura via SSH, garantindo a primeira bandeira (*User Flag*) em `/home/robert/user.txt`.

3.2 Path Injection (Exploração de SUID)

Identificamos um binário chamado `bugtracker` com a permissão **SUID** (executa com privilégios do dono, no caso, o *root*). A vulnerabilidade reside no fato do programa chamar o comando `cat` sem especificar seu caminho absoluto (como `/bin/cat`).

Como estudantes de computação, sabemos que o sistema busca comandos seguindo a ordem da variável **$PATH**. Se criarmos um executável falso com o nome `cat` e o colocarmos em um diretório que venha primeiro na lista de busca, o programa executará o nosso script em vez do comando real.

```
# Criando um script que spawna um shell
echo "/bin/sh" > /tmp/cat
chmod +x /tmp/cat

# Alterando o PATH para que o sistema procure no /tmp primeiro
export PATH=/tmp:$PATH

# Executando o programa vulner vel
/usr/bin/bugtracker
```

Listing 3: Exploração de Path Injection

Ao rodar o `bugtracker`, ele executou o nosso `cat` (que na verdade era um shell) com privilégios de root.

4 Conclusão

A exploração da máquina **Oopsie** demonstra que falhas simples em aplicações web podem escalar para o comprometimento total do servidor se as permissões de sistema e variáveis de ambiente não forem configuradas com o princípio do menor privilégio.